

[Home](#) [News and blogs hub](#)

Lost and Frustrated but Persistent, part 1: personal narratives about usability challenges with Open Source Scientific Software



Bookmark this page

Lost and Frustrated but Persistent, part 1: personal narratives about usability challenges with Open Source Scientific Software

Author(s)



M. Doherty SSI fellow

A. Eggert

Y. Eid

K. Lehmann

C. Meesters

L. Schüler

D. Wicaksono

Posted on 3 July 2023

Estimated read time: 9 min

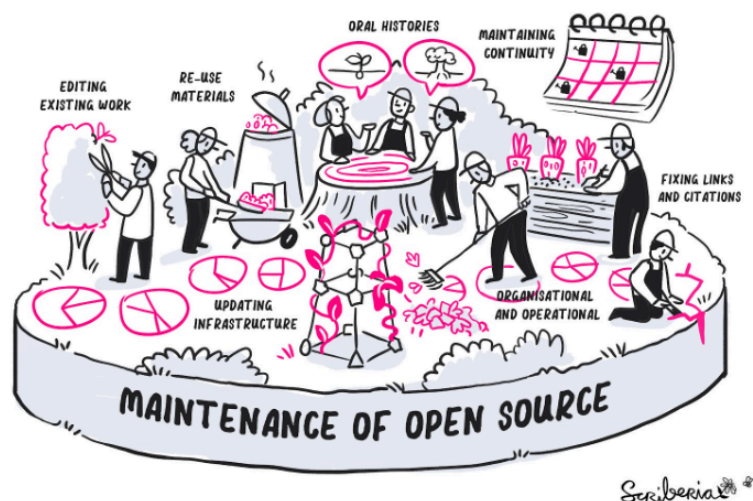
Sections in this article

The Lost Early-Career Researcher

The Accidental Research Software Engineer



Posted by d.barclay on 3 July 2023 - 8:30am



This illustration is created by Scriberia with The Turing Way community. Used under a CC-BY 4.0 licence. DOI: 10.5281/zenodo.3332807

By SSI Fellow Meag Doherty, Anja Eggert, Yomna Eid, Kjong-Van Lehmann, Christian Meesters, Lennart Schüler, Damar Wicaksono.

Over the course of four days earlier in April, a group of us at the **Open Science Retreat** spent time discussing **challenges and opportunities with usability in Open Source Scientific Software**.

As a way to wrap up the time spent together, a few group members wrote personal narratives (including self-assigned catchy titles!) that highlight some of the individual and collective experiences in usability.

The narrative structure included the following questions:

- What is your role and context?
- What is the core issue regarding usability in your daily work?
- What are the critical barriers you need to overcome? (or maybe you don't know yet!)
- Who can help you and the community over this critical barrier?

The Lost Early-Career Researcher

As an early career researcher, I have primarily had the experience of using pre-existing packages and integrating them into the workflows I am developing for my analysis with spatio-temporal data derived from multiple resources.

Having only tinkered around with building my own package, I am lost in a flurry of information. I would like to create workflows that are guaranteed to still run in the future, but it is not uncommon to experience errors with broken scripts due to the package dependencies. Generally, I try to steer away from unstable packages, but one isn't so lucky all the time. I also have a hard time incorporating certain functions from individual packages into my workflow, on more than one occasion, as sufficient documentation is hard to come by all the time. I am less inclined to use such a package and seek alternatives. **The best packages to use for me are ones that go a step further and provide a test example of how certain functions work.** It is always easier to learn by example rather than by text alone.

And if I magically happen to get so far as to generalize my algorithm and workflows into complete ready-to-go packages that represent my methodology, **I do not know where to start to ensure that someone beyond me would use it and benefit from it.** I need concrete guidelines on what makes up the robustness and usability of the package and the type, amount and extent of thorough documentation. To me, it seems arbitrary most times and a function of personal taste rather than a standard. It doesn't help that sometimes these standards differ across domains/communities.

This compilation of issues could easily discourage early scientists (likely scientists, in general) from openly sharing their code, as the lack of best practices guidelines in combination with the lack of acknowledgement of the effort that goes into it is a high investment but low reward. I discovered that there exists a gap between my ambitions and training, that they do not align perfectly to meet their goals. That is partially due to **the ambiguity of what the scientific community up to this point defines as good software/packages, even though we know a good one when we see one.** Without the formal training of a software developer, I ask myself what could be done to make this journey easier and less complicated than it seems?

The Accidental Research Software Engineer

I'm an RSE coming from substantive research, using software as a medium to make the methods I develop more accessible to fellow scientists and students. For me, that frequently means creating graphical user interfaces that reduce the reliance on code and coding skills.

I don't have a formal background in usability, so **while I care deeply about making the software useful and approachable, I am limited to haphazard attempts and intuitive solutions.** The bulk of my user feedback comes from teaching students, running workshops, observing colleagues and answering their questions, so I rarely get to observe entirely naive first-time users systematically. My impression is that **(my) interfaces are designed around the underlying technical design of the software, and may not serve users' interests in the best possible way.**

I wish there were institutional support for usability research and improvement. My contributions are largely evaluated by their scientific merit and substantive feature set, and **I can't always justify spending time on interfaces alone without expanding functionality.** This does not mean that researchers don't care about usability - to the contrary, I believe that usable tools are a great way to overcome barriers and pave the road towards better research practices, and that better-designed software has a larger impact. However, this is not reflected in the current incentive system.

I would love there to be design- and usability-related guidance for the entirety of the research software lifecycle. Starting early on, feedback could help shape the tools and interfaces we build, and better align them with scientists' needs without having to scrap and overhaul existing work. Later, it could point out areas for improvement, and help successful open source tools compete with proprietary alternatives. Throughout, I believe this would be most helpful if it could keep in mind the scientific goals and resource constraints in RSE work, and support developers and researchers in implementing the proposals, possibly by including the user community. Ideally, rather than merely adding to the list of requirements for a research software tool, this could help reduce the support load that many RSEs currently face.

The Pragmatic Bioinformatics Researcher

As a Bioinformatician in an academic setting, I often analyze high-dimensional data using the latest state-of-the-art methods. The decision on which method to use is driven by the expected novelty of the results, the overall performance, matching assumptions and plausible theoretical or algorithmic foundation but not usability. At the same time, we also strive to contribute novel approaches. Both tasks involve running similar methods to decide on the best method to apply or to benchmark the novel approach against existing methods.

Getting other researchers' methods to run is a time-consuming process that already stalls at installation procedures and dependency resolutions, input formatting, usage examples, documentation and versioning. While the situation overall has slowly improved, in some cases, the underlying method and published results convincingly suggest a superior performance although the accompanying software is unusable. This sometimes requires to be pragmatic and re-implement the approach from scratch.

In a research setting, the final product is not the software but the dissemination of the novelty (e.g., publication). **Therefore, there is little reward for good software engineering and long-term maintenance.**

Maintaining relevant software packages past publication takes away time from future research and often, the developer has moved on. Also, Bioinformatics Researchers can write code, but they are usually not trained software engineers. However, their contributions in terms of methods and models from this community have arguably played a part in getting biomedical research to where it is now. The dependency on these contributions coupled with software development without software engineering support probably plays a large role in the current state of research software.

Adding **training may alleviate the situation.** But how much software engineering expertise should we be expecting from a researcher? Providing researchers with software engineering support could be an alternative approach. While it seems like an additional expense to an institution, overall productivity and quality may significantly improve.

The Eagerly Support-Guy

“Working on a High Performance Compute Cluster? - Well, if I have to ...” The lament is long and ranges from overly bureaucratic access regulations to various technical details rendering one’s research on an official cluster impossible. I overheard this HPC-no-thank-you remark frequently in hallway tracks. And working to support cluster users, I have to admit: **Usability and HPC clusters do not go along well.**

Adapting life science workflows is particularly challenging. A single such “workflow” (designed to analyze some sort of data, e.g. genetic material) may require several dozen applications. All of these software tools need to be deployed, I/O-issues need to be solved. Some applications clearly are “runs on my system”-only software packages. Not all HPC clusters allow using “**Bioconda**”, a package management systems tailored for the life sciences which can be used by any user to install needed software. As a consequence, a frustrated user base (“No, we cannot support *your* software.”) is inevitable.

This HPC attitude forces entire user groups to build and maintain their own computing base. As a result, we see a waste of money and resources poured into redundant infrastructure.

Is there any silver lining?

In order to meet user expectations, **the HPC community mindset has to change.** Away from **chasing FLOPS and providing ever faster computers**, towards the attempt to support all scientific domains. This would require to reach out and listen. We would stop providing “**true HPC**” clusters and gain a pleased user base.

Continues in part.

Share on blog/article:



[Subscribe](#)

[Data management](#)

[Privacy policy](#)

[Web accessibility](#)

[Environmental Sustainability Policy](#)

[Contact](#)

[Site map](#)



©2010 - 2026 The University of
Edinburgh on behalf of the Software
Sustainability Institute.

Except where otherwise noted,
content on this site is licensed under
a Creative Commons Attribution
Non-Commercial 2.5 License.